Metody numeryczne
Interpolacja
IO/IT, WIMiIP

Danuta Szeliga

AGH Kraków

2025/2026

Spis treści I

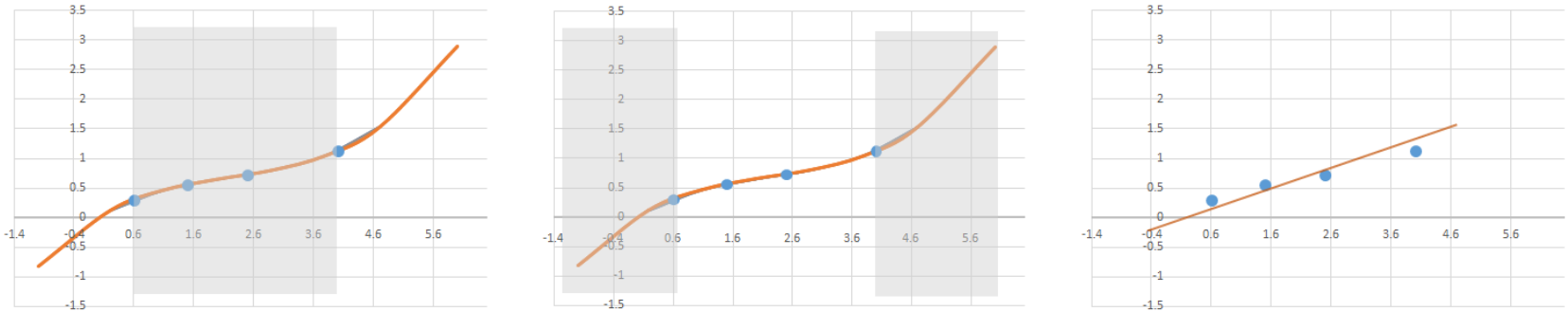
1. Wstęp
2. Interpolacja
3. Postacie wielomianu
   - Postać naturalna
   - Postać Newtona
   - Wielomiany Czebyszewa
4. Interpolacja Lagrange'a
   - Definicja wielomianu
   - Wyznaczanie wartości
   - Ilorazy różnicowe - węzły jednokrotne
   - Interpolacja Lagrange'a w przypadku węzłów równoodległych
5. Interpolacja Hermite'a
   - Sformułowanie zadania
   - Postać Newtona
   - Ilorazy różnicowe
   - Zbieżność procesu interpolacyjnego
6. Interpolacja trygonometryczna
   - Sformułowanie zadania
   - Wyznaczanie współczynników
   - Szybkie transformacje Fouriera FFT
7. Interpolacja funkcjami sklejanymi

Spis treści II

8 Przykłady
- Interpolacja Lagrange'a
- Reszta interpolacyjna
- Interpolacja Hermite'a

Pojęcia podstawowe

- *Interpolacja*. Funkcja, np. wielomian stopnia n, przechodzi dokładnie przez zadane punkty. Poszukiwane są wartości pomiędzy zadanymi węzłami.
- *Ekstrapolacja*. Funkcja, np. wielomian stopnia n, przechodzi dokładnie przez zadane punkty. Poszukiwane są wartości poza zakresem węzłów.
- *Aproksymacja*. Funkcja, np. wielomian stopnia mniejszego niż n, dopasowany do n zadanych punktów wg przyjętego kryterium dopasowania.



Interpolacja, ekstrapolacja, aproksymacja

Interpolacja

- Modelowanie zjawisk, dla których nieznane są funkcje opisujące zależności pomiędzy obserwowanymi/mierzalnymi wielkościami

Interpolacja

- Modelowanie zjawisk, dla których nieznane są funkcje opisujące zależności pomiędzy obserwowanymi/mierzalnymi wielkościami
- Dostępne są informacje o wartościach lub pochodnych tych funkcji dla określonych argumentów (danych wejściowych) - dane dyskretne

Interpolacja

- Modelowanie zjawisk, dla których nieznane są funkcje opisujące zależności pomiędzy obserwowanymi/mierzalnymi wielkościami
- Dostępne są informacje o wartościach lub pochodnych tych funkcji dla określonych argumentów (danych wejściowych) - dane dyskretne
- Przypadek ogólny - funkcja dana poprzez wartości pewnych funkcjonałów $\mathcal{F}_j$ na funkcji f, tj.

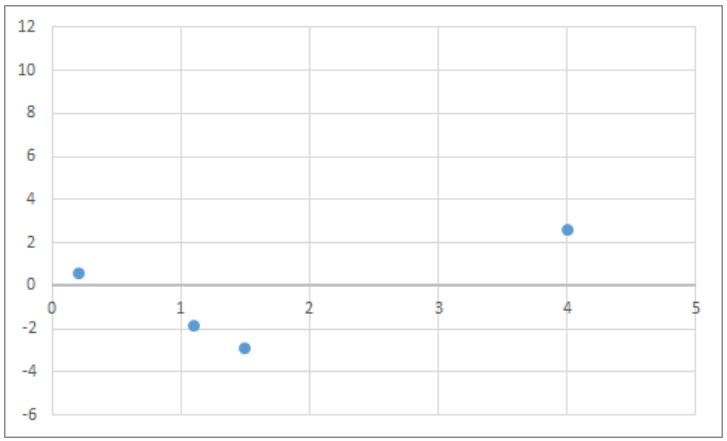
$$\mathfrak{R}(f) = \{\mathcal{F}_j(f) : \quad j = 1, \ldots, n\}$$

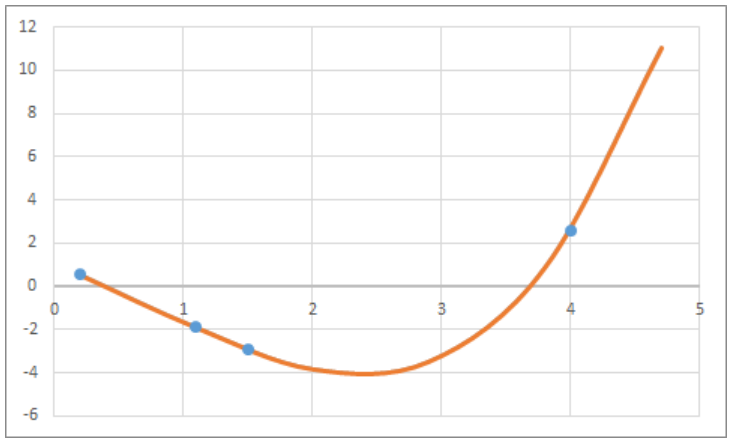
Interpolacja

Zadanie interpolacyjne polega na znalezieniu w ustalonej klasie funkcji "przybliżenia" g danej funkcji f, dla którego funkcjonały $\mathfrak{R}$ przyjmują te same wartości

$$\mathcal{F}_j(f) = \mathcal{F}_j(g) \quad dla\ j = 0, 1, \ldots, n \quad tzn.\ \mathfrak{R}(f) = \mathfrak{R}(g)$$

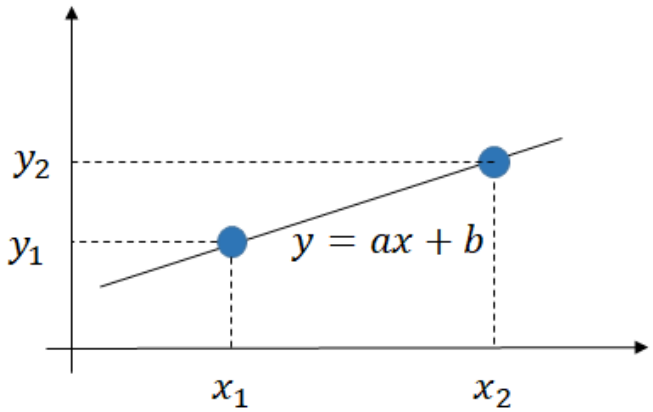
Przykład
Przypadek najprostszy - interpolacja wielomianowa, gdy dany jest dyskretny zbiór punktów





Najprostsze zadanie interpolacji I

Znajdź liniową zależność $y = ax + b$, a i b są nieznane, pomiędzy dwoma punktami (x_1, y_1) orax (x_2, y_2)



Stosując zapis macierzowy

$$\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

Rozwiązanie: dowolna metoda rozwiązywania liniowych układów równań, np. metodą wyznaczników

$$a = (y_2 - y_1)/(x_2 - x_1) \qquad b = (x_2 y_1 - x_1 y_2)/(x_2 - x_1)$$

Postać Newtona

$$y = y_1 + \frac{(y_2 - y_1)}{(x_2 - x_1)}(x - x_1)$$

Postać Lagrange'a

$$y = \frac{(x - x_2)}{(x_1 - x_2)} y_1 + \frac{(x - x_1)}{(x_2 - x_1)} y_2$$

Przykład

Interpolacja okręgu na podstawie zadanej liczby punktów
Okrąg opisany jest krzywą parametryczną postaci:

$$\begin{cases} x(t) = \cos(t\pi/2) \\ y(t) = \sin(t\pi/2) \end{cases}$$

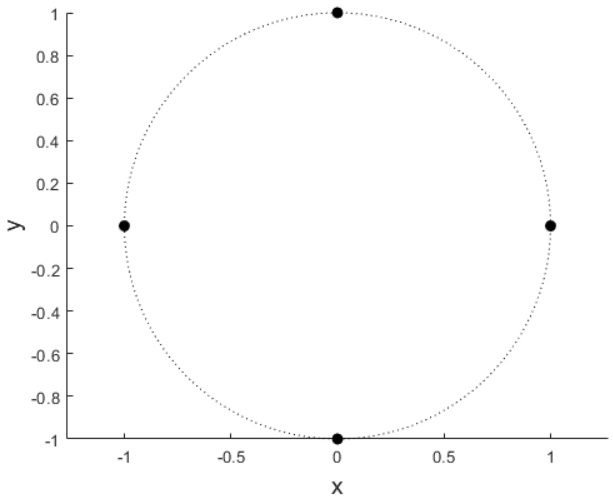
Okrąg będzie interpolowany dla następujących wartości:

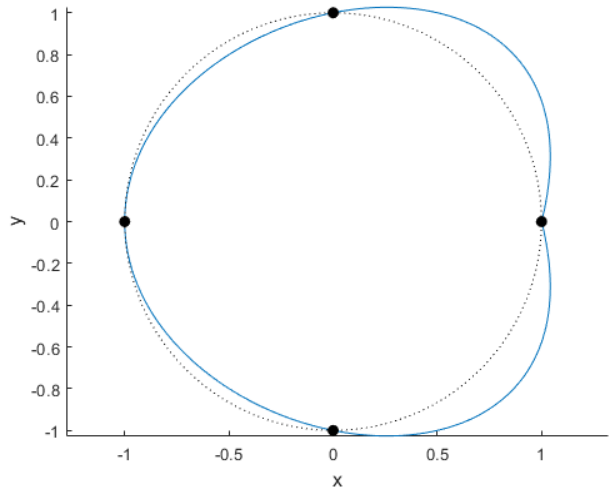
t	0	1	2	3	4
x	1	0	-1	0	1
y	0	1	0	-1	0

Tworzymy układy równań $\mathbf{Ta} = \mathbf{x}$ (cosinus) i $\mathbf{Tb} = \mathbf{y}$ (sinus), gdzie

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 & 16 \\ 1 & 3 & 9 & 27 & 81 \\ 1 & 4 & 16 & 64 & 256 \end{bmatrix}$$

Rozwiązujemy: $\mathbf{a} = \mathbf{T}^{-1}\mathbf{x}$, $\mathbf{b} = \mathbf{T}^{-1}\mathbf{y}$

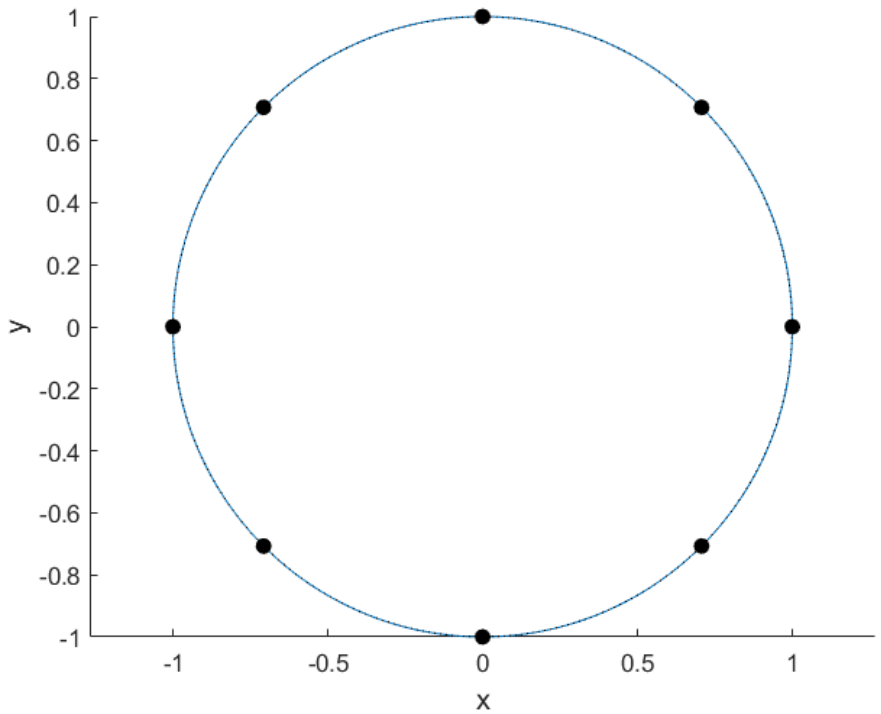




Przykład - cd.

Poprawa jakości rozwiązania $\rightarrow$ dołożenie dodatkowych punktów

t	0	0.5	1	1.5	2	2.5	3	3.5	4
x	1	0.7071	0	-0.7071	-1	-0.7071	0	0.7071	1
y	0	0.7071	1	0.7071	0	0.7071	-1	-0.7071	0



Postacie wielomianu

Postacie wielomianu

Wielomian w postaci naturalnej (rozwinięcie potęgowe)

$$w(x) = \sum_{k=0}^{n} a_k x^k$$

Postacie wielomianu

Wielomian w postaci naturalnej (rozwinięcie potęgowe)

$$w(x) = \sum_{k=0}^{n} a_k x^k$$

Reprezentacja wymaga zapamiętania w pamięci $n+1$ współczynników a_k

Postacie wielomianu

Wielomian w postaci naturalnej (rozwinięcie potęgowe)

$$w(x) = \sum_{k=0}^{n} a_k x^k$$

Reprezentacja wymaga zapamiętania w pamięci $n+1$ współczynników a_k

Postać naturalna jest rozwinięciem Maclaurina wielomanu w, gdzie $a_k = \frac{w^{(k)}(0)}{k!}$
Informacja o wielomianie stopnia n składa się z jego znormalizowanych pochodnych w punkcie zero.

Obliczenie wartości wielomianu $w(x)$ w punkcie x, $x \neq 0$

Wielomian w postaci naturalnej można przedstawić w postaci:

$$w(x) = (\ldots(a_n x + a_{n-1})x + \ldots + a_1)x + a_0$$

i stąd wynika *schemat Hornera* obliczania wartości wielomianu $w(x)$:

$$\begin{aligned} w_n &= a_n \\ w_i &= w_{i+1} \cdot x + a_i \quad dla\ i = n-1, n-2, \ldots, 0 \end{aligned}$$

Oczywiste jest, że $w(x) = w_0$

Można pokazać, że algorytm Hornera jest numerycznie poprawny (wskaźnik kumulacji co najwyżej $2n+1$)

Postacie wielomianu

Wielomian w postaci Newtona

$$w(x) = \sum_{k=0}^{n} b_k p_k(x)$$

gdzie

$$p_0(x) \stackrel{df}{=} 1$$
$$p_k(x) \stackrel{df}{=} (x - x_0)(x - x_1) \cdots (x - x_{k-1}) \quad dla\ k = 1, 2, \ldots, n$$

a x_0, x_1, ..., x_n są danymi liczbowymi.

Postacie wielomianu

Wielomian w postaci Newtona

$$w(x) = \sum_{k=0}^{n} b_k p_k(x)$$

gdzie

$$p_0(x) \stackrel{df}{=} 1$$
$$p_k(x) \stackrel{df}{=} (x - x_0)(x - x_1) \cdots (x - x_{k-1}) \quad dla\ k = 1, 2, \ldots, n$$

a x_0, x_1, ..., x_n są danymi liczbowymi.

Wielomiany p_k $(k = 0, 1, \ldots, n)$ tworzą bazę przestrzeni W_n wielomianów stopnia nie wyższego niż n

Postacie wielomianu

Wielomian w postaci Newtona można zapisać jako:

$$w(x) = (\ldots(b_n(x - x_{n-1}) + b_{n-1})(x - x_{n-2}) + \ldots + b_1)(x - x_0) + b_0$$

Postacie wielomianu

Wielomian w postaci Newtona można zapisać jako:

$$w(x) = (\ldots(b_n(x - x_{n-1}) + b_{n-1})(x - x_{n-2}) + \ldots + b_1)(x - x_0) + b_0$$

Jeżeli zdefiniujemy rekurencyjnie wielomiany (*uogólniony schemat Hornera*):

$$\begin{aligned} w_n &= b_n \\ w_i &= w_{i+1} \cdot (x - x_i) + b_i \quad \textit{dla } i = n-1, n-2, \ldots, 0 \end{aligned}$$

to otrzymamy

$$w(x) = w_0(x)$$

Postacie wielomianu

Wyznaczenie współczynników postaci naturalnej dla wielomianu w postaci Newtona

Każdy z wielomianów pomocniczych zdefiniowanych w uogólnionym algorytmie Hornera zapisujemy w postaci naturalnej:

$$w_i(x) = \sum_{k=i}^{n} a_k^{(i)} x^{k-i} \quad i = n, n-1, \ldots, 0$$

i z rekurencyjnej definicji otrzymujemy:

$$\begin{aligned} a_n^{(n)} &= b_n \\ w_i(x) &= \sum_{k=i+1}^{n} a_k^{(i+1)} x^{k-i} - \sum_{k=i+1}^{n} a_k^{(i+1)} x^{k-(i+1)} x_i + b_i \\ &= a_n^{(i+1)} x^{n-i} + \sum_{k=i+1}^{n-1} \left(a_k^{(i+1)} - a_{k+1}^{(i+1)} x_i \right) x^{k-i} + \left(b_i - a_{i+1}^{(i+1)} x_i \right) \end{aligned}$$

Postacie wielomianu

Wyznaczenie współczynników postaci naturalnej dla wielomianu w postaci Newtona - cd

Stąd współczynniki wielomianu postaci naturalnej kolejnych wielomianów w_i $(i = n, n-1, \ldots, 0)$ można wyznaczyć rekurencyjnie:

$$\begin{aligned} a_n^{(i)} &= b_n \\ a_i^{(i)} &= b_i - a_{i+1}^{(i+1)} x_i \\ a_k^{(i)} &= a_k^{(i+1)} - a_{k+1}^{(i+1)} x_i \qquad k = i+1, i+2, \ldots, n-i \end{aligned}$$

A szukane współczynniki a_k są równe $a_k^{(0)}$ $(k = 0, 1, \ldots, n)$.

Koszt algorytmu to $\frac{n(n+1)}{2}$ mnożeń i odejmowań.

Postacie wielomianu

Wielomian w postaci kombinacji liniowych wielomianów ortogonalnych
Przykład - rozwinięcie względem wielomianów Czebyszewa pierwszego rodzaju T_k, zdefiniowane rekurencyjnie:

$$
\begin{aligned}
T_0(x) &= 1 \\
T_1(x) &= x \\
T_k(x) &= 2xT_{k-1}(x) - T_{k-2}(x) \quad \textit{dla } k = 2, 3, \ldots
\end{aligned}
$$

Postacie wielomianu

Wielomian w postaci kombinacji liniowych wielomianów ortogonalnych
Przykład - rozwinięcie względem wielomianów Czebyszewa pierwszego rodzaju T_k, zdefiniowane rekurencyjnie:

$$\begin{aligned} T_0(x) &= 1 \\ T_1(x) &= x \\ T_k(x) &= 2xT_{k-1}(x) - T_{k-2}(x) \quad dla\ k = 2, 3, \ldots \end{aligned}$$

Wielomiany te tworzą bazę $\rightarrow$ dowolny wielomian w można przedstawić jednoznacznie w postaci

$$w(x) = \sum_{k=0}^{n} c_k T_k(x)$$

Postacie wielomianu

Algorytm obliczania wartości wielomianu $w(x)$ w punkcie x:

$$
\begin{aligned}
& w(x) = \\
& = c_n T_n(x) + c_{n-1} T_{n-1}(x) + c_{n-2} T_{n-2}(x) + \sum_{k=0}^{n-3} c_k T_k(x) \\
& = (2x\tilde{c}_n + c_{n-1} - \tilde{c}_{n+1}) T_{n-1}(x) + (c_{n-2} - \tilde{c}_n) T_{n-2}(x) + \sum_{k=0}^{n-3} c_k T_k(x) \\
& \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \\
& = (2x\tilde{c}_2 + c_1 - \tilde{c}_3) T_1(x) + (c_0 - \tilde{c}_2) T_0(x)
\end{aligned}
$$

gdzie

$$
\begin{aligned}
& \tilde{c}_{n+2} = \tilde{c}_{n+1} = 0 \\
& \tilde{c}_k = 2x\tilde{c}_{k+1} + c_k - \tilde{c}_{k+2} \qquad dla\ k = n, n-1, \ldots, 1
\end{aligned}
$$

Przejście do postaci naturalnej.

Interpolacja Lagrange'a

Sformułowanie zadania

Zadanie interpolacyjne Lagrange'a polega na znalezieniu dla danej funkcji f^* wielomianu L_n stopnia nie wyższego niż n, którego wartości w $n+1$ punktach x_i są takie same jak wartości interpolowanej funkcji:

$$L_n(x_i) = f(x_i) \quad dla\ i = 0, 1, \ldots, n$$

gdzie $x_i \neq x_j$ dla $i \neq j$.

Konstrukcja wielomianu

Zadanie interpolacyjne Lagrange'a ma jednoznaczne rozwiązanie, które można przedstawić w postaci:

$$L_n(x) = \sum_{i=0}^{n} f(x_i) l_i(x)$$

gdzie

$$l_i(x) \stackrel{df}{=} \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

Konstrukcja wielomianu

Zadanie interpolacyjne Lagrange'a ma jednoznaczne rozwiązanie, które można przedstawić w postaci:

$$L_n(x) = \sum_{i=0}^{n} f(x_i) l_i(x)$$

gdzie

$$l_i(x) \stackrel{df}{=} \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

l_i to wielomiany stopnia n takie, że

$$l_i(x_j) = \delta_{ij} = \begin{cases} 1 & dla\ i = j \\ 0 & dla\ i \neq j \end{cases}$$

Przykład wielomianu interpolacyjnego Lagrange'a

Wielomiany bazowe L_i, dla $i = 0 \ldots 3$, dla węzłów $x_0 = -2$, $x_1 = -1$, $x_2 = 0$, $x_3 = 1$, $x_4 = 2$

$$L_4(x) = y_0 L_0(x) + y_1 L_1(x) + \\ + y_2 L_2(x) + y_3 L_3(x)$$

$$L_0(x) = \frac{(x - x_1)(x - x_2)(x - x_3)(x - x_4)}{(x_0 - x_1)(x_0 - x_2)(x_0 - x_3)(x_0 - x_4)}$$

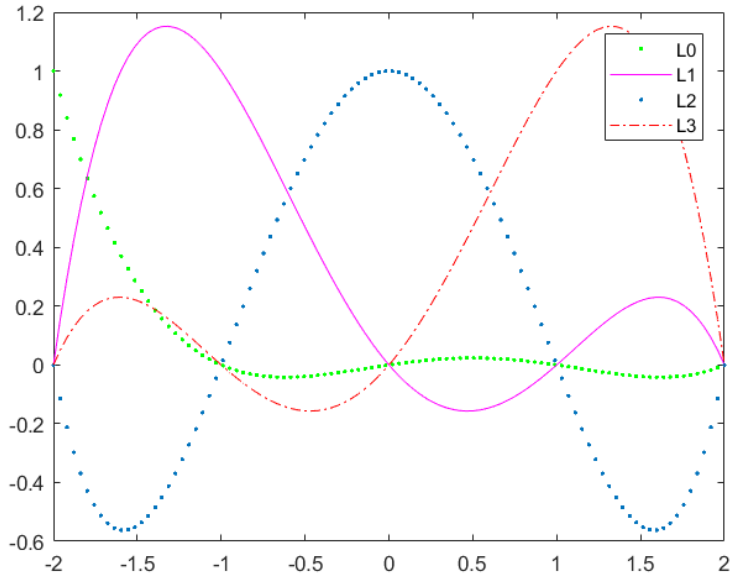
$$L_1(x) = \frac{(x - x_0)(x - x_2)(x - x_3)(x - x_4)}{(x_1 - x_0)(x_1 - x_2)(x_1 - x_3)(x_1 - x_4)}$$

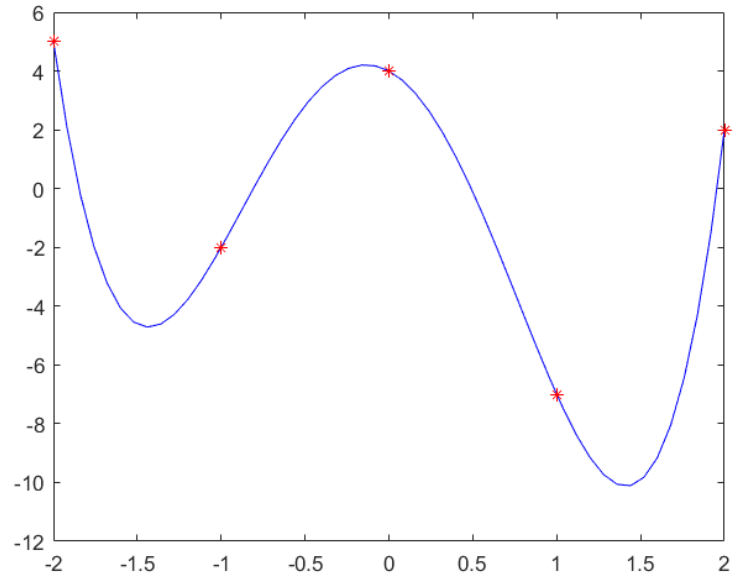
. . . . . .

I przykładowy wielomian L_4 dla $y_0 = 5$, $y_1 = -2$, $y_2 = 4$, $y_3 = -7$, $y_4 = 2$

Zauważmy, że dla dowolnego x zachodzi: $\sum_{i=0}^{n} L_i(x) = 1$

czyli w wielomianie interpolacyjnym każdy wielomian L_i ma udział wagowy równy y_i





Interpolacja Lagrange'a

Obliczanie wartości wielomianu $L_n(x)$ lub jego pochodnej w danym punkcie x na podstawie jego definicji nie jest wygodne.

Można zmiejszyć koszt obliczeniowy i pamięciowy jeśli przedstawimy wielomian $L_n(x)$ w postaci naturalnej ze współczynnikami a_k:

$$L_n(x) = \sum_{k=0}^{n} a_k x^k$$

Jak wyznaczyć te współczynniki?

Interpolacja Lagrange'a

Obliczanie wartości wielomianu $L_n(x)$ lub jego pochodnej w danym punkcie x na podstawie jego definicji nie jest wygodne.

Można zmiejszyć koszt obliczeniowy i pamięciowy jeśli przedstawimy wielomian $L_n(x)$ w postaci naturalnej ze współczynnikami a_k:

$$L_n(x) = \sum_{k=0}^{n} a_k x^k$$

Jak wyznaczyć te współczynniki?

Należy rozwiązać układ równań liniowych postaci:

$$a_0 + a_1 x_i + \ldots + a_n x_i^n = f(x_i) \qquad i = 0, 1, \ldots, n$$

Interpolacja Lagrange'a

Układ równań $\sum_{k=0}^{n} a_k x_i^k = f(x_i)$ w wielu przypadkach, np. dla węzłów równoodległych, jest b. źle uwarunkowany ze względu na macierz współczynników $X = (x_i^{j-1}) \rightarrow$ specjalne metody rozwiązywania układów tego typu albo ...

Interpolacja Lagrange'a

Układ równań $\sum_{k=0}^{n} a_k x_i^k = f(x_i)$ w wielu przypadkach, np. dla węzłów równoodległych, jest b. źle uwarunkowany ze względu na macierz współczynników $X = (x_i^{j-1}) \rightarrow$ specjalne metody rozwiązywania układów tego typu albo ...

wyznaczamy najpierw współczynniki b_k postaci Newtona wielomianu $L_n(x)$:

$$L_n(x) = \sum_{k=0}^{n} b_k p_k(x)$$

a następnie przechodzimy do postaci naturalnej.

Koszt obliczeniowy tych operacji to n^2 działań arytmetycznych (koszt obliczeniowy rozwiązania liniowego układu równań standardową metodą to n^3.)

Postać Newtona wielomianu Lagrange'a

Dla wyznaczenia współczynników b_k zauważmy, że jeżeli L_{k-1} i L_k są wielomianami interpolacyjnymi funkcji f opartymi na węzłach $x_0, x_1, \ldots, x_{k-1}$ i $x_0, x_1, \ldots, x_k$, to spełniają zależność rekurencyjną:

$$L_k(x) = L_{k-1}(x) + b_k p_k(x)$$

Każdy z wielomianów p_k ma współczynnik 1 przy x^k (z definicji tych wielomianów), a stąd wynika, że współczynnik b_k jest równy współczynnikowi przy x^k w wielomianie L_k.

Postać Newtona wielomianu Lagrange'a

Pamiętając, że wielomian L_k jest postaci (z definicji):

$$L_k(x) = \sum_{i=0}^{k} f(x_i) \prod_{\substack{j=0 \\ j \neq i}}^{k} \frac{x - x_j}{x_i - x_j}$$

możemy obliczyć współczynniki b_k:

$$b_k = \sum_{i=0}^{k} \frac{f(x_i)}{\prod_{\substack{j=0 \\ j \neq i}}^{k} (x_i - x_j)}$$

Postać Newtona wielomianu Lagrange'a

Pamiętając, że wielomian L_k jest postaci (z definicji):

$$L_k(x) = \sum_{i=0}^{k} f(x_i) \prod_{\substack{j=0 \\ j \neq i}}^{k} \frac{x - x_j}{x_i - x_j}$$

możemy obliczyć współczynniki b_k:

$$b_k = \sum_{i=0}^{k} \frac{f(x_i)}{\prod_{\substack{j=0 \\ j \neq i}}^{k} (x_i - x_j)}$$

Współczynniki b_k to ilorazy różnicowe funkcji f oparte na węzłach $x_0, x_1, \ldots, x_k$.

Ilorazy różnicowe dla węzłów jednokrotnych

Węzły jednokrotne to węzły spełniające warunek $x_i \neq x_j$ dla $i \neq j$.

Ilorazy różnicowe dla węzłów jednokrotnych

Węzły jednokrotne to węzły spełniające warunek $x_i \neq x_j$ dla $i \neq j$.

Definicja. Ilorazem różnicowym rzędu k funkcji f opartym na parami różnych węzłach $x_l, x_{l+1}, \ldots, x_{l+k}$, w których określona jest funkcja f, nazywamy wyrażenie $f[x_l, x_{l+1}, \ldots, x_{l+k}]$ postaci:

$$f[x_l, x_{l+1}, \ldots, x_{l+k}] = \sum_{i=l}^{l+k} \frac{f(x_i)}{\prod\limits_{\substack{j=l \\ j \neq i}}^{l+k} (x_i - x_j)}$$

Ilorazy różnicowe dla węzłów jednokrotnych

Węzły jednokrotne to węzły spełniające warunek $x_i \neq x_j$ dla $i \neq j$.

Definicja. Ilorazem różnicowym rzędu k funkcji f opartym na parami różnych węzłach $x_l, x_{l+1}, \ldots, x_{l+k}$, w których określona jest funkcja f, nazywamy wyrażenie $f[x_l, x_{l+1}, \ldots, x_{l+k}]$ postaci:

$$f[x_l, x_{l+1}, \ldots, x_{l+k}] = \sum_{i=l}^{l+k} \frac{f(x_i)}{\prod\limits_{\substack{j=l \\ j \neq i}}^{l+k} (x_i - x_j)}$$

Przez iloraz różnicowy rzędu zerowego oparty na węźle x_l rozumiana jest wartość funkcji w tym punkcie, tzn. $f[x_l] = f(x_l)$.

Ilorazy różnicowe dla węzłów jednokrotnych

Można pokazać, że dla dowolnego układu parami różnych punktów $x_l, x_{l+1}, \ldots, x_{l+k}$ należących do dziedziny funkcji f, zachodzi zależność rekurencyjna:

$$f\left[x_l, x_{l+1}, \ldots, x_{l+k}\right] = \frac{f\left[x_{l+1}, x_{l+1}, \ldots, x_{l+k}\right] - f\left[x_l, x_{l+1}, \ldots, x_{l+k-1}\right]}{x_{l+k} - x_l}$$

Ilorazy różnicowe dla węzłów jednokrotnych

Można pokazać, że dla dowolnego układu parami różnych punktów $x_l, x_{l+1}, \ldots, x_{l+k}$ należących do dziedziny funkcji f, zachodzi zależność rekurencyjna:

$$f\left[x_l, x_{l+1}, \ldots, x_{l+k}\right] = \frac{f\left[x_{l+1}, x_{l+1}, \ldots, x_{l+k}\right] - f\left[x_l, x_{l+1}, \ldots, x_{l+k-1}\right]}{x_{l+k} - x_l}$$

Zauważmy, że współczynniki postaci Newtona są równe $b_k = f\left[x_0, x_1, \ldots, x_k\right]$.

Ilorazy różnicowe dla węzłów jednokrotnych

Współczynniki postaci Newtona b_k można wyznaczyć obliczając rekurencyjnie odpowiednie ilorazy różnicowe.
Dla węzłów x_i i wartości $f(x_i)$ tworzymy trójkątną tablicę:

$$
\begin{array}{llll}
x_0 & f(x_0) & & \\
x_1 & f(x_1) & f[x_0,x_1] = \frac{f(x_1)-f(x_0)}{x_1-x_0} & \\
x_2 & f(x_2) & f[x_1,x_2] = \frac{f(x_2)-f(x_1)}{x_2-x_1} & f[x_0,x_1,x_2] = \frac{f[x_1,x_2]-f[x_0,x_1]}{x_2-x_0} \\
\vdots & \vdots & \vdots & \ddots \\
x_n & f(x_n) & f[x_{n-1},x_n] & f[x_0,\ldots,x_n] = \frac{f[x_1,\ldots,x_n]-f[x_0,\ldots,x_{n-1}]}{x_n-x_0}
\end{array}
$$

Ilorazy różnicowe dla węzłów jednokrotnych

Współczynniki postaci Newtona b_k można wyznaczyć obliczając rekurencyjnie odpowiednie ilorazy różnicowe.
Dla węzłów x_i i wartości $f(x_i)$ tworzymy trójkątną tablicę:

$$\begin{array}{llll}
x_0 & f(x_0) & & \\
x_1 & f(x_1) & f[x_0,x_1]=\frac{f(x_1)-f(x_0)}{x_1-x_0} & \\
x_2 & f(x_2) & f[x_1,x_2]=\frac{f(x_2)-f(x_1)}{x_2-x_1} & f[x_0,x_1,x_2]=\frac{f[x_1,x_2]-f[x_0,x_1]}{x_2-x_0} \\
\vdots & \vdots & \vdots & \ddots \\
x_n & f(x_n) & f[x_{n-1},x_n] & f[x_0,\ldots,x_n]=\frac{f[x_1,\ldots,x_n]-f[x_0,\ldots,x_{n-1}]}{x_n-x_0}
\end{array}$$

Współczynniki b_k są równe elementom przekątniowym tablicy, tzn. $b_k = f[x_0, x_1, \ldots, x_k]$.
Algorytm ilorazów różnicowych wymaga $n(n-1)/2$ dzieleń i dwa razy więcej odejmowań.

Interpolacja Lagrange'a w przypadku węzłów równoodległych

Często węzły interpolacyjne są równoodległe (i rzeczywiste):

$$x_i = x_0 + ih \quad dla\ i = 0, 1, \ldots, n$$

gdzie h to stała długość kroku. Wielomian Lagrange'a ma teraz postać:

$$L_n(x) = L_n(x_0 + th) = \sum_{i=0}^{n} f(x_i) \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{t-j}{i-j}$$

Interpolacja Lagrange'a w przypadku węzłów równoodległych

Przy węzłach równoodległych wygodna jest postać Newtona wielomianu interpolacyjnego, której współczynniki wyznaczane są za pomocą różnic skończonych funkcji f:
Różnica progresywna rzędu k funkcji f to

$$\begin{aligned} \Delta^0 f(x) &= f(x) \\ \Delta^k f(x) &= \Delta^{k-1} f(x+h) - \Delta^{k-1} f(x) \qquad k = 1, 2, \ldots \end{aligned}$$

Różnica wsteczna rzędu k to

$$\begin{aligned} \nabla^0 f(x) &= f(x) \\ \nabla^k f(x) &= \nabla^{k-1} f(x) - \nabla^{k-1} f(x-h) \qquad k = 1, 2, \ldots \end{aligned}$$

Interpolacja Lagrange'a w przypadku węzłów równoodległych

Można pokazać, że jeżeli $x_i = x_0 + ih$ dla $i = 0, 1, \ldots, n$ to

$$f[x_0, x_1, \ldots, x_k] = \frac{\Delta^k f(x_0)}{k! h^k}$$

Interpolacja Lagrange'a w przypadku węzłów równoodległych

Można pokazać, że jeżeli $x_i = x_0 + ih$ dla $i = 0, 1, \ldots, n$ to

$$f[x_0, x_1, \ldots, x_k] = \frac{\Delta^k f(x_0)}{k! h^k}$$

Dla $x = x_0 + th$ postać wielomianów p_k jest następująca:

$$p_k(x) = p_k(x_0 + th) = h^k \prod_{i=0}^{k-1} (t - i)$$

Interpolacja Lagrange'a w przypadku węzłów równoodległych

Można pokazać, że jeżeli $x_i = x_0 + ih$ dla $i = 0, 1, \ldots, n$ to

$$f[x_0, x_1, \ldots, x_k] = \frac{\Delta^k f(x_0)}{k! h^k}$$

Dla $x = x_0 + th$ postać wielomianów p_k jest następująca:

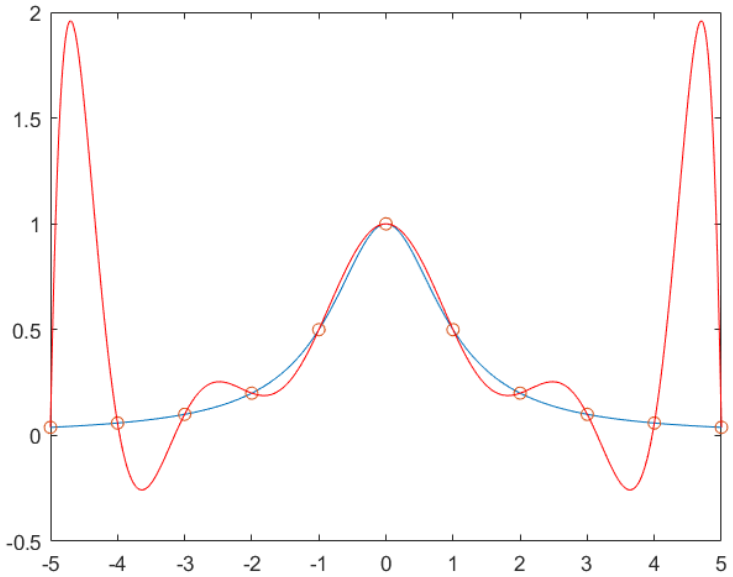
$$p_k(x) = p_k(x_0 + th) = h^k \prod_{i=0}^{k-1} (t - i)$$

Przyjmując oznaczenie $q_k(t) := \prod_{i=0}^{k-1} (t - i)$ i korzystając z zależości $f[x_0, x_1, \ldots, x_k] = \frac{\Delta^k f(x_0)}{k! h^k}$, można zapisać wielomian interpolacyjny Lagrange'a jako:

$L_n(x_0 + th) = \Delta^0 f(x_0) + \Delta f(x_0) q_1(t) + \frac{\Delta^2 f(x_0)}{2!} q_2(t) + \ldots + \frac{\Delta^n f(x_0)}{n!} q_n(t)$

Uwagi do interpolacji Lagrange'a

- Interpolacja dla dużej liczby węzłów: otrzymujemy wielomiany wysokiego stopnia
- Szczególnie dla węzłów równoodległych prowadzi do oscylacji na końcach przedziału
- Zadanie interpolacji wielomianem wysokiego stopnia jest też wrażliwe na zaburzenie danych, jest źle uwarunkowane numerycznie
- Stosowana jest albo interpolacja lokalna niższego stopnia, czyli funkcje sklejane, albo interpolacja z narzuconymi węzłami Czebyszewa



Efekt Rungego dla funkcji $y = 1/(1 + x^2)$

Interpolacja Hermite'a

Interpolacja Hermite'a

Danych jest $k+1$ różnych węzłów $x_0, x_1, \ldots, x_k$ oraz liczby naturalne $m_0, m_1, \ldots, m_k$ takie, że $\sum_{i=0}^{k} m_i = n+1$. Zadanie interpolacyjne Hermite'a polega na znalezieniu dla danej funkcji f^* wielomianu H_n stopnia nie wyższego niż n, spełniającego warunki:

$$H_n^{(j)}(x_i) = f^{(j)}(x_i) \qquad i = 0, 1, \ldots k \quad j = 0, 1, \ldots, m_i - 1$$

Interpolacja Hermite'a

Danych jest $k+1$ różnych węzłów $x_0, x_1, \ldots, x_k$ oraz liczby naturalne $m_0, m_1, \ldots, m_k$ takie, że $\sum_{i=0}^{k} m_i = n+1$. Zadanie interpolacyjne Hermite'a polega na znalezieniu dla danej funkcji f^* wielomianu H_n stopnia nie wyższego niż n, spełniającego warunki:

$$H_n^{(j)}(x_i) = f^{(j)}(x_i) \qquad i = 0, 1, \ldots k \quad j = 0, 1, \ldots, m_i - 1$$

W szczególnym przypadku, gdy $m_i = 1$ interpolacja Hermite'a jest równoważna zadaniu interpolacji Lagrange'a.

Interpolacja Hermite'a - postać Newtona wielomianu

Wprowadźmy oznaczenie:

$$s(i) := \begin{cases} 0 & i = 0 \\ m_0 + m_1 + \ldots + m_{i-1} & i > 0 \end{cases}$$

gdzie $s(i)$ oznacza sumę krotności i początkowych węzłów interpolacyjnych.
Każda liczba l, $0 \leqslant l \leqslant n$, daje się jednoznacznie przedstawić w postaci $l = s(i) + j$, gdzie $0 \leqslant i \leqslant k$ i $0 \leqslant j \leqslant m_i - 1$.

Interpolacja Hermite'a - postać Newtona wielomianu

Wprowadźmy oznaczenie:

$$s(i) := \begin{cases} 0 & i = 0 \\ m_0 + m_1 + \ldots + m_{i-1} & i > 0 \end{cases}$$

gdzie $s(i)$ oznacza sumę krotności i początkowych węzłów interpolacyjnych.
Każda liczba l, $0 \leqslant l \leqslant n$, daje się jednoznacznie przedstawić w postaci $l = s(i) + j$, gdzie $0 \leqslant i \leqslant k$ i $0 \leqslant j \leqslant m_i - 1$.

Definiujeny wielomiany $p_{s(i)+j}$:

$$\begin{aligned} p_{s(0)}(x) &:= 1 \\ p_{s(i)+j}(x) &:= (x - x_0)^{m_0}(x - x_1)^{m_1} \cdots (x - x_{i-1})^{m_{i-1}}(x - x_i)^j \\ & \qquad\qquad i = 0, 1, \ldots, k \quad j = 0, 1, \ldots, m_i - 1 \end{aligned}$$

Postać Newtona wielomianu - cd

Szukany wielomian interpolacyjny H_n zapisujemy w postaci Newtona jako:

$$H_n(x) = \sum_{l=0}^{n} b_l p_l(x) = \sum_{i=0}^{k} \sum_{j=0}^{m_i-1} b_{s(i)+j} p_{s(i)+j}(x)$$

Współczynniki b_l będą wyznaczane kolejno dla $l = 0, 1, \ldots, n$.

Ilorazy różnicowe

Podobnie jak dla interpolacji Lagrange'a, współczynniki b_l postaci Newtona są równe odpowiednim ilorazom różnicowym.
Najprostszy przypadek - iloraz różnicowy funkcji f oparty na dwukrotnym węźle x_i.
Zakładamy, że f jest dwukrotnie różniczkowalna w x_i i otrzymujemy:

$$f[x_i, x_i] := \lim_{x_{i+1} \to x_i} \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} = f'(x_i)$$

Ilorazy różnicowe

Podobnie jak dla interpolacji Lagrange'a, współczynniki b_l postaci Newtona są równe odpowiednim ilorazom różnicowym.
Najprostszy przypadek - iloraz różnicowy funkcji f oparty na dwukrotnym węźle x_i.
Zakładamy, że f jest dwukrotnie różniczkowalna w x_i i otrzymujemy:

$$f[x_i, x_i] := \lim_{x_{i+1} \to x_i} \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} = f'(x_i)$$

Dla zapisu ilorazów różnicowych opartych na węzłach wielokrotnych stosujemy oznaczenia:

$$f[x_l, i_l; x_{l+1}, i_{l+1}; \ldots; x_{l+k}, i_{l+k}] = \\ = f[\underbrace{x_l, \ldots, x_l}_{i_l \text{ razy}}, \underbrace{x_{l+1}, \ldots, x_{l+1}}_{i_{l+1} \text{ razy}}, \ldots, \underbrace{x_{l+k}, \ldots, x_{l+k}}_{i_{l+k} \text{ razy}}]$$

Ilorazy różnicowe - cd

Definicja. Ilorazy różnicowe funkcji f oparte na wielokrotnych węzłach definiujemy jako zależności

- dla i-krotnego węzła x_l

$$f[x_l, i] = \frac{f^{(i-1)}(x_l)}{(i-1)!}$$

- dla różnych węzłów $x_l, x_{l+1}, \ldots, x_{l+k}$ o krotnościach odpowiednio $i_l, i_{l+1}, \ldots, i_{l+k}$

$$\begin{aligned} f[x_l, i_l; x_{l+1}, i_{l+1}; \ldots; x_{l+k}, i_{l+k}] = \\ (f[x_l, i_l - 1; x_{l+1}, i_{l+1}; \ldots; x_{l+k}, i_{l+k}] \\ - f[x_l, i_l; x_{l+1}, i_{l+1}; \ldots; x_{l+k}, i_{l+k} - 1]) \,/\, (x_{l+k} - x_l) \end{aligned}$$

Zakładamy, że istnieją pochodne $f^{(j)}(x_{i_{l+m}})$ dla $m = 0, 1, \ldots, k$, $j = 0, 1, \ldots, i_{l+m} - 1$.

Ilorazy różnicowe - cd

Współczynniki b_l, $(l = s(i) + j,\ i = 0, 1, \ldots, k,\ j = 0, 1, \ldots, m_i - 1)$ postaci Newtona dla wielomianu Hermite'a są równe ilorazom różnicowym interpolowanej funkcji opartym na początkowych węzłach z uwzględnieniem ich krotności:

$$b_l = f[x_0, m_0; x_1, m_1; \ldots; x_{i-1}, m_{i-1}; x_i, j + 1]$$

Ilorazy różnicowe - cd

Współczynniki b_l, $(l = s(i) + j,\ i = 0, 1, \ldots, k,\ j = 0, 1, \ldots, m_i - 1)$ postaci Newtona dla wielomianu Hermite'a są równe ilorazom różnicowym interpolowanej funkcji opartym na początkowych węzłach z uwzględnieniem ich krotności:

$$b_l = f[x_0, m_0; x_1, m_1; \ldots; x_{i-1}, m_{i-1}; x_i, j+1]$$

Można pokazać, że jeżeli rzeczywista funkcja f ma ciągłą $(n+1)$-szą pochodną na odcinku $[a, b]$ zawierającym węzły rzeczywiste $x_0, x_1, \ldots, x_k, x$, to istnieje punkt ξ, $\xi \in [a, b]$, dla którego

$$f[x_0, m_0; x_1, m_1; \ldots; x_k, m_k; x] = \frac{f^{(n+1)}(\xi)}{(n+1)!}$$

Ilorazy różnicowe - cd

Co więcej z zależności:

$$f[x_0, m_0; x_1, m_1; \ldots; x_k, m_k; x] = \frac{f^{(n+1)}(\xi)}{(n+1)!}$$

oraz różnicy progrestywnej dla węzłów równoodległych:

$$f[x_0, x_1, \ldots x_k] = \frac{\Delta^k f(x_0)}{k! h^k}$$

wynika, że jeżeli f jest wielomianem stopnia nie większego niż n, to ilorazy różnicowe i różnice skończone f rzędu wyższego od n są równe zeru.

Ilorazy różnicowe - cd

Co więcej z zależności:

$$f[x_0, m_0; x_1, m_1; \ldots; x_k, m_k; x] = \frac{f^{(n+1)}(\xi)}{(n+1)!}$$

oraz różnicy progrestywnej dla węzłów równoodległych:

$$f[x_0, x_1, \ldots x_k] = \frac{\Delta^k f(x_0)}{k! h^k}$$

wynika, że jeżeli f jest wielomianem stopnia nie większego niż n, to ilorazy różnicowe i różnice skończone f rzędu wyższego od n są równe zeru.

Ilorazy różnicowe - przykład

Przykład. Dane są węzły x_0 i x_1 o krotnościach odpowiednio $m_0 = 3$, $m_1 = 2$.
Dla danej funkcji f szukamy wielomianu interpolacyjnego Hermite'a H_4 takiego, że:

$$H_4(x_0) = f(x_0) \qquad H_4'(x_0) = f'(x_0) \qquad H_4''(x_0) = f''(x_0)$$
$$H_4(x_1) = f(x_1) \qquad H_4'(x_1) = f'(x_1)$$

Wielomian H_4 można zapisać w postaci Newtona:

$$H_4(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)^2 + b_3(x - x_0)^3 + b_4(x - x_0)^3(x - x_1)$$

Współczynniki b_i są odpowiednimi ilorazami różnicowymi leżącymi na przekątnej, np.
$b_0 = f[x_0] = f(x_0)$, $b_3 = f[x_0, 3; x_1]$

Ilorazy różnicowe - przykład, cd.

Tablica ilorazów różnicowych dla omawianego przykładu jest postaci

$$
\begin{array}{llllll}
x_0 & f(x_0) & & & & \\
x_0 & f(x_0) & f[x_0,2]=f'(x_0) & & & \\
x_0 & f(x_0) & f[x_0,2]=f'(x_0) & f[x_0,3]=\frac{f''(x_0)}{2!} & & \\
x_1 & f(x_1) & f[x_0,x_1]=\frac{f(x_1)-f(x_0)}{x_1-x_0} & f[x_0,2;x_1]=\frac{f[x_0,x_1]-f[x_0,2]}{x_1-x_0} & f[x_0,3;x_1] & \\
x_1 & f(x_1) & f[x_1,2]=f'(x_1) & f[x_0;x_1,2]=\frac{f[x_1,2]-f[x_0,x_1]}{x_1-x_0} & f[x_0,2;x_1,2]=\frac{f[x_0;x_1,2]-f[x_0,2;x_1]}{x_1-x_0} & f[x_0,3;x_1,2]=\frac{f[x_0,2;x_1,2]-f[x_0,3;x_1]}{x_1-x_0}
\end{array}
$$

Reszta wzoru interpolacyjnego

Reszta interpolacyjna:

$$r(x) = f(x) - H_n(x)$$

Z warunków interpolacyjnych wiemy, że węzły x_i są m_i-krotymi zerami funkcji r. Stąd:

$$r(x) = p_{n+1}(x)g(x)$$

gdzie $p_{n+1}(x) = (x - x_0)^{m_0}(x - x_1)^{m_1} \cdots (x - x_k)^{m_k}$. Funkcję g można określić tak, aby była ciągła na przedziale $[a, b]$ (można udowodnić stosowane twierdzenie).

Reszta wzoru interpolacyjnego

Jeżli funkcja f jest $(m_i - 1)$-krotnie różniczkowalna w punktach x_i, $i = 0, 1, \ldots, k$, to dla $x \neq x_i$ zachodzi

$$r(x) = p_{n+1}(x) f[x_0, m_0; x_1, m_1; \ldots; x_k, m_k; x]$$

Reszta wzoru interpolacyjnego

Jeżli funkcja f jest $(m_i - 1)$-krotnie różniczkowalna w punktach x_i, $i = 0, 1, \ldots, k$, to dla $x \neq x_i$ zachodzi

$$r(x) = p_{n+1}(x) f[x_0, m_0; x_1, m_1; \ldots; x_k, m_k; x]$$

Co więcej, można pokazać, że jeżeli funkcja f ma ciągłą $(n+1)$-szą pochodną na odcinku $[a, b]$ zawierającym węzły rzeczywiste x_i, $i = 0, \ldots, k$ i punkt x, to istnieje $\xi = \xi(x)$, $\xi \in [a, b]$, dla którego

$$r(x) = p_{n+1}(x) \frac{f^{(n+1)}(\xi)}{(n+1)!}$$

Wzór interpolacyjny Newtona

Z definicji wielomianów Hermite'a,współczynników b_i wielomianu Hermite'a w postaci Newtona oraz postaci reszty interpolacyjnej wynika zależność zwana *wzorem interpolacyjnym Newtona*:

$$
\begin{aligned}
f(x) = &\sum_{i=0}^{k} \sum_{j=0}^{m_i-1} f[x_0, m_0; x_1, m_1; \ldots; x_{i-1}, m_{i-1}; x_i, j+1] \\
&\times (x-x_0)^{m_0}(x-x_1)^{m_1} \cdots (x-x_{i-1})^{m_{i-1}}(x-x_i)^{j} \\
&+ \underbrace{f[x_0, m_0; x_1, m_1; \ldots; x_k, m_k; x](x-x_0)^{m_0}(x-x_1)^{m_1} \cdots (x-x_k)^{m_k}}_{r(x)}
\end{aligned}
$$

Wzór interpolacyjny Newtona - cd

Dla interpolacji Lagrange'a wzór interpolacyjny Newtone upraszcza się do postaci:

$$f(x) = \sum_{i=0}^{n} f[x_0, x_1, \ldots x_i](x - x_0)(x - x_1)\cdots(x - x_{i-1}) + f[x_0, x_1, \ldots x_n](x - x_0)(x - x_1)\cdots(x - x_n)$$

Zbieżność ciągów wielomianów przy $n \to \infty$

Przykład 1. Można pokazać, że ciąg $\{L_n\}$ wielomianów interpolacyjnych funkcji f opartych na węzłach równooległych z przedziału $[a, b]$ jest jednostajnie zbieżny do f na tym przedziale.

Zbieżność ciągów wielomianów przy $n \to \infty$

Przykład 1. Można pokazać, że ciąg $\{L_n\}$ wielomianów interpolacyjnych funkcji f opartych na węzłach równooległych z przedziału $[a, b]$ jest jednostajnie zbieżny do f na tym przedziale.

Przykład 2 (Rungego). Niech L_n będzie wielomianem interpolacyjnym funkcji $f(x) = \frac{1}{1+x^2}$, opartym na równoodległych węzłach z przedziału $[-5, 4]$, $x_i = -5 + ih$, $i = 0, 1, \ldots, n$, $h = \frac{10}{n}$. Funkcja f jest klasy $C^{\infty}_{[-5,5]}$, a ciąg $\{L_n\}$ jest zbieżny do $f(x)$ tylko dla $|x| < 3.63$ i rozbieżny dla pozostałych wartości x.

Zbieżność - cd

Prawdziwe są następujące twierdzenia

Faber

Dla dowolnego układu węzłów interpolacyjnych istnieje funkcja ciągła, do której ciąg wielomianów interpolacyjnych nie jest jednostajnie zbieżny.

Z drugiej strony....

Dla dowolnej funkcji f ciągłej na odcinku $[a, b]$ istnieje ciąg wielomianów interpolacyjnych zbieżnych do f jednostajnie na $[a, b]$.

Nie jest to twierdzenie konstruktywne ....

Interpolacja trygonometryczna

Interpolacja trygonometryczna. Sformułowanie zadania

Interpolacja funkcji okresowych, czyli takich, że:

$$g(y + t) = g(y)$$

dla wszystkich y.
Zmieniając zmienną $x = \frac{2\pi}{t} y$ otrzymujemy funkcję okresową o okresie 2π:

$$f(x) = g\left(\frac{yt}{2\pi}\right)$$

Będziemy rozważać, bez straty ogólności, tylko funkcje o okresie 2π.

Sformułowanie zadania - cd

Zakładamy, że f jest funkcją zmiennej rzeczywistej o wartościach zespolonych.
W takim przypadku wygodnie jest rozważać wielomian, który jest kombinacją liniową zespolonych funkcji wykładniczych postaci:

$$e^{i\,jx} = \cos jx + i \sin jx \quad i = \sqrt{-1}$$

Sformułowanie zadania - cd

Zakładamy, że f jest funkcją zmiennej rzeczywistej o wartościach zespolonych.
W takim przypadku wygodnie jest rozważać wielomian, który jest kombinacją liniową zespolonych funkcji wykładniczych postaci:

$$e^{i\,jx} = \cos jx + i \sin jx \quad i = \sqrt{-1}$$

Zadanie interpolacji trygonometrycznej jest równoznaczne ze znalezieniem dla danej funkcji f wielomianu trygonometrycznego:

$$t_n(x) = \sum_{j=0}^{n} c_j e^{i\,jx}$$

który w $n+1$ różnych punktach x_k z przedziału $[0, 2\pi]$ przyjmuje te same wartości co interpolowana funkcja:

$$t_n(x_k) = f(x_k) \quad k = 0, 1, \ldots, n \quad x_k \neq x_l \; dla \; k \neq l$$

Wyznaczanie współczynników wielomianu trygonometrycznego

Będziemy rozważać wyznaczanie współczynników wielomianu trygonometrycznego dla węzłów równoodległych:

$$x_k = \frac{2k\pi}{n+1} \qquad k = 0, 1, \ldots, n$$

Przy tym założeniu rozwiązanie zadania interpolacyjnego istotnie się upraszcza.

Wyznaczanie współczynników - cd

Funkcje $e^{i\,jx}$, $j = 0, 1, \ldots, n$, tworzą układ ortogonalny w sensie iloczynu skalarnego:

$$(f, g) = \sum_{k=0}^{n} f(x_k)\bar{g}(x_k)$$

Można sprawdzić, że:

$$\left(e^{i\,lx}, e^{i\,jx}\right) = \sum_{k=0}^{n} e^{i\,lx} e^{-i\,jx} = \sum_{k=0}^{n} e^{i(\,l-j)\frac{2k\pi}{n+1}x} =$$

$$= \begin{cases} \frac{e^{i(\,l-j)2\pi}-1}{e^{i(\,l-j)\frac{2\pi}{n+1}}-1} = 0 & dla\ l \neq j \\ n+1 & dla\ l = j \end{cases}$$

Wyznaczanie współczynników - cd

Korzystając z informacji o iloczynie skalarnym, wyznaczamy wpółczynniki wielomianu trygonometrycznego $t_n(x) = \sum_{j=0}^{n} c_j e^{i\,jx}$ interpolującego funkcję f, opartego na węzłach $x_k = \frac{2k\pi}{n+1}$, $k = 0, 1, \ldots, n$, które są postaci:

$$c_j = \frac{\left(f, e^{i\,jx}\right)}{n+1} = \frac{1}{n+1} \sum_{k=0}^{n} f(x_k) e^{-i\,jx_k}$$

Wyznaczanie współczynników - cd

Korzystając z informacji o iloczynie skalarnym, wyznaczamy wpółczynniki wielomianu trygonometrycznego $t_n(x) = \sum_{j=0}^{n} c_j e^{i\,jx}$ interpolującego funkcję f, opartego na węzłach $x_k = \frac{2k\pi}{n+1}$, $k = 0, 1, \ldots, n$, które są postaci:

$$c_j = \frac{\left(f, e^{i\,jx}\right)}{n+1} = \frac{1}{n+1} \sum_{k=0}^{n} f(x_k) e^{-i\,jx_k}$$

Współczynniki c_j są równe współczynnikom *rozwinięcia Fouriera* funkcji f względem iloczynu skalarnego, a zadanie wyznaczania tych współczynników - *dyskretną analizą Fouriera*.

Wyznaczanie współczynników dla funkcji o wartościach rzeczywistych

Trygonometryczny wielomian interpolacyjny funkcji f, oparty na węzłach $x_k = \frac{2k\pi}{n+1}$, $k = 0, 1, \ldots, n$, można przedstawić w sposób równoważny do definicji $t_n(x) = \sum_{j=0}^{n} c_j e^{i\,jx}$ w postaci:

$$t_n(x) = \frac{1}{2}a_0 + \sum_{j=1}^{m} (a_j \cos jx + b_j \sin jx) + \delta \frac{1}{2} a_{m+1} \cos(m+1)x \tag{1}$$

gdzie $\begin{array}{ll} \delta = 0, & m = \frac{1}{2}n \quad gdy\ n\ parzyste \\ \delta = 1, & m = \frac{1}{2}(n-1) \quad gdy\ n\ nieparzyste \end{array}$

współczynniki a_j, b_j są równe

$$\begin{aligned} a_j &= \tfrac{2}{n+1} \sum_{k=0}^{n} f(x_k) \cos jx_k \\ b_j &= \tfrac{2}{n+1} \sum_{k=0}^{n} f(x_k) \sin jx_k \end{aligned} \tag{2}$$

Powyższe współ. dla funkcji f o wart.rzeczywistych są też rzeczywiste.

Szybkie transformacje Fouriera FFT

Algorytm wyznaczania współczynników c_j
lub
Rozwiązywanie zadania odwrotnego - obliczania wartości wielomianu trygonometrycznego w punktach $x_k = \frac{2k\pi}{n+1}$, $k = 0, 1, \ldots, n$. Przyjmujemy oznaczenie $f(k) := f(x_k)$

- ZA - analiza Fouriera: dla danych liczb zespolonych $f(0), f(1), \ldots, f(k)$ szukamy wielkości

$$c(j) = \frac{1}{n+1} \sum_{k=0}^{n} f(k)\, e^{-i\, j \frac{2\pi k}{n+1}} \qquad j = 0, 1, \ldots, n$$

- ZS - synteza Fouriera (zadanie odwrotne do ZA): mając dane liczby zespolone $c(0), c(1), \ldots, c(n)$ szukamy

$$f(k) = \sum_{j=0}^{n} c(j)\, e^{i\, j \frac{2\pi k}{n+1}} \qquad k = 0, 1, \ldots, n$$

FFT. Algorytm

- Klasyczny algorytm rozwiązania ZA i ZS wymaga ok. $(n+1)^2$ zespolonych mnożeń i dodawań oraz obliczania $e^{-i\,j\frac{2\pi k}{n+1}}$
- Algorytm Cooleya i Tukeya szybkich transformacji Fouriera (FFT) jest obliczeniowo tańszy.
  Jeżli założymy, że $n+1 = r_1 r_2 \ldots r_p$, to liczba zespolonych mnożeń i dodawań w alg. FFT jest rzędu $(n+1)(r_1 + r_2 + \ldots + r_p)$.
  Najczęściej FFT stosuje się dla $n+1 = 2^k$ (czyli wszystkich $r_j = 2$) i wtedy alg. wymaga $n \log_2 n$ rzędu działań.

FFT. Algorytm wg Reinscha

Wprowadzamy wielkości pomocnicze:

$$P_0 := 1, \qquad P_j := \prod_{k=1}^{j} r_k \quad dla\ j = 1, 2, \ldots, p$$

$$Q_j := \prod_{k=j+1}^{p} r_k \quad dla\ j = 0, 1, \ldots, p-1, \qquad Q_p := 1$$

Zauważmy, że dowolną liczbę całkowitą z przedziału $[0, n]$ można jednoznacznie przedstawić w postaci:

$$j = j_p P_{p-1} + \ldots + j_1 P_0 \qquad gdzie\ 0 \leqslant j_i \leqslant r_i - 1 \quad dla\ i = 1, 2, \ldots, p$$

lub

$$k = k_1 Q_1 + \ldots + k_p Q_p \qquad gdzie\ 0 \leqslant k_i \leqslant r_i - 1 \quad dla\ i = 1, 2, \ldots, p$$

FFT. Algorytm wg Reinscha - cd

Wzór na współczynniki $c(j)$ dla węzłów równoodległych można więc zapisać jako:

$$c(j) = \frac{1}{n+1} \sum_{k_p=0}^{r_p-1} \dots \sum_{k_1=0}^{r_1-1} e^{-2\pi ij(k_1 Q_1 + \dots + k_p Q_p)/(n+1)} f(k_1 Q_1 + \dots + k_p Q_p) \tag{3}$$

Wiemy, że $Q_i P_i = n+1$, $i = 0, 1, \dots, p$, wobec tego:

$$(k_1 Q_1 + \dots + k_p Q_p)/(n+1) = \frac{k_1}{P_1} + \dots + \frac{k_p}{P_p} \tag{4}$$

FFT. Algorytm wg Reinscha - cd

Z równań (3) i (4) wynika zależność:

$$
\begin{aligned}
(n+1)c(j) = & \sum_{k_p=0}^{r_p-1} e^{-2\pi i\, j \frac{k_p}{P_p}} \\
& \sum_{k_{p-1}=0}^{r_{p-1}-1} e^{-2\pi i\, j \frac{k_{p-1}}{P_{p-1}}} \\
& \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \\
& \sum_{k_1=0}^{r_1-1} e^{-2\pi i\, j \frac{k_1}{P_1}} f(k_1 Q_1 + \ldots + k_p Q_p)
\end{aligned}
$$

Ponadto dla $m = 1, 2, \ldots, p$ mamy:

$$
e^{-2\pi i\, j \frac{k_m}{P_m}} = e^{-2\pi i\, (j_p P_{p-1} + \ldots + j_1 P_0) \frac{k_m}{P_m}} = e^{-2\pi i\, (j_m P_{m-1} + \ldots + j_1 P_0) \frac{k_m}{P_m}}
$$

FFT. Algorytm wg Reinscha - cd

Ostatecznie więc otrzymujemy:

$$
\begin{aligned}
(n+1)c(j) &= (n+1)c(j_p P_{p-1} + \ldots + j_1 P_0) = \\
&= \sum_{k_p=0}^{r_p-1} e^{-2\pi i\,(j_p P_{p-1}+\ldots+j_1 P_0)\frac{k_p}{P_p}} \\
&\quad \sum_{k_{p-1}=0}^{r_{p-1}-1} e^{-2\pi i\,(j_{p-1} P_{p-2}+\ldots+j_1 P_0)\frac{k_{p-1}}{P_{p-1}}} \\
&\quad \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \\
&\quad {}_1\Big\{ \sum_{k_1=0}^{r_1-1} e^{-2\pi i\, j_1 P_0 \frac{k_1}{P_1}} \\
&\quad\quad {}_0\big\{ f(k_1 Q_1 + \ldots + k_p Q_p)
\end{aligned}
$$

Wyrażenie zakreślone klamrą ${}_0\{$ oznaczamy symbolem $S_0(k_1 Q_1 + \ldots + k_p Q_p)$, klamrą ${}_1\{$ symbolem $S_1(j_1 Q_1 + k_2 Q_2 + \ldots + k_p Q_p)$, ..., klamrą ${}_p\{$ symbolem $S_p(j_1 Q_1 + \ldots + j_p Q_p)$.

FFT. Algorytm Cooleya i Tukeya

Przy oznaczeniach jak na poprzednim slajdzie, algorytm Cooleya i Tukeya jest postaci:

- dla $k_i = 0, 1, \ldots, r_i - 1,\ i = 1, 2, \ldots, p$

$$S_0(k_1 Q_1 + \ldots + k_p Q_p) := f(k_1 Q_1 + \ldots + k_p Q_p)$$

- dla $m = 1, 2, \ldots, p$

$$\begin{aligned} &S_m(j_1 Q_1 + \ldots j_m Q_m + k_{m+1} Q_{m+1} + \ldots + k_p Q_p) \\ &:= \sum_{k_m=0}^{r_m-1} e^{-2\pi i (j_m P_{m-1} + \ldots + j_1 P_0) \frac{k_m}{P_m}} \\ &\cdot S_{m-1}(j_1 Q_1 + \ldots j_{m-1} Q_{m-1} + k_m Q_m + \ldots + k_p Q_p) \end{aligned}$$

gdzie $j_i = 0, 1, \ldots, r_i - 1$ dla $i = 1, 2, \ldots, m$,
$k_j = 0, 1, \ldots, r_j - 1$ dla $j = m + 1, m + 2, \ldots, p$.

FFT. Algorytm Cooleya i Tukeya - cd

Wprowadźmy dodatkowe oznaczenia:

$$a := k_{m+1}Q_{m+1} + \ldots + k_pQ_p$$
$$b := j_{m-1}P_{m-2} + \ldots + j_1P$$
$$\bar{b} := j_1Q_1 + \ldots j_{m-1}Q_{m-1}$$

oraz

$$u := e^{-2\pi i/P_m} \qquad v := e^{-2\pi i/r_m}$$

A zatem w m-tym kroku:

$$\begin{aligned} S_m(\bar{b} + j_mQ_m + a) &= \sum_{k_m=0}^{r_m-1} e^{-2\pi i\left(\frac{j_m}{r_m}+\frac{b}{P_m}\right)k_m} S_{m-1}(\bar{b} + k_mQ_m + a) \\ &= \sum_{k_m=0}^{r_m-1} v^{j_mk_m}u^{bk_m}S_{m-1}(\bar{b} + k_mQ_m + a) \end{aligned}$$

dla $b = 0, 1, \ldots, P_{m-1} - 1$, $a = 0, 1, \ldots, Q_m - 1$, $j_m = 0, 1, \ldots, r_m - 1$.

FFT. Algorytm Cooleya i Tukeya - cd

Można więc dla $m = 1, 2, \ldots, p$ algorytm realizować następująco:
dla $b = 0, 1, \ldots, P_{m-1} - 1$
dla $a = 0, 1, \ldots, Q_m - 1$

- skalowanie, czyli

$$dla \quad k = 0, 1, \ldots r_m - 1 \qquad S'_{m-1}(\bar{b} + kQ_m + a) := u^{bk} S_{m-1}(\bar{b} + kQ_m + a)$$

- transformacja, czyli

$$dla \quad j = 0, 1, \ldots r_m - 1 \qquad S_{m-1}(\bar{b} + jQ_m + a) := \sum_{k=0}^{r_m - 1} v^{jk} S'_{m-1}(\bar{b} + kQ_m + a)$$

FFT. Algorytm Cooleya i Tukeya - cd

Skalowanie i transformacja są równoważne mnożeniu odpowiednio przez macierze **D** i **T** postaci:

$$\mathbf{D} = \begin{bmatrix} 1 & 0 & 0 & . & . & 0 \\ 0 & u^b & 0 & . & . & 0 \\ 0 & 0 & u^{2b} & . & . & 0 \\ . & . & . & . & . & . \\ . & . & . & . & . & 0 \\ 0 & 0 & . & . & . & u^{(r_m-1)b} \end{bmatrix} \qquad \mathbf{T} = \begin{bmatrix} 1 & 1 & . & . & . & 1 \\ 1 & v & v^2 & . & . & v^{r_m-1} \\ . & v^2 & v^4 & . & . & . \\ . & . & . & . & . & . \\ . & . & . & . & . & . \\ 1 & v^{r_m-1} & v^{2(r_m-1)} & . & . & v^{(r_m-1)^2} \end{bmatrix}$$

i wymagają łącznie około r_m^2 mnożeń i tyle samo dodawań. Ogółem w m-tym kroku algorytmu trzeba wykonać $O(P_{m-1}Q_m r_m^2)$ działań.
Ponieważ $P_{m-1}Q_m = \frac{n+1}{r_m}$, to w m-tym kroku jest $O((n+1)r_m)$ działań. Wobec tego koszt całego algorytmu jest równy $O((n+1)(r_1 + r_2 + \ldots + r_p)$ zespolonych dodawań i mnożeń.

Algorytm FFT dla funkcji o wartościach rzeczywistych I

- Można wykorzystać alg. Cooleya i Tukeya do wyznaczenia rzeczywistych współczynników a_j i b_j (równania (2)) trygonometrycznego wielomianu interpolacyjnego w postaci (1) lub
- do obliczenia jego wartości w punktach $x_k = \frac{k\pi}{n+1}$, $k = 0, 1, \ldots, 2n+1$.

Odpowiednikiem ZA i ZS są w tym wypadku zadania:

- RA - mając dane liczby rzeczyw. y_k, $k = 0, 1, \ldots, 2n+1$ szukamy wielkości

$$a_j = \frac{1}{n+1} \sum_{k=0}^{2n+1} y_k \cos \frac{\pi jk}{n+1} \qquad j = 0, 1, \ldots, n+1$$

oraz

$$b_j = \frac{1}{n+1} \sum_{k=0}^{2n+1} y_k \sin \frac{\pi jk}{n+1} \qquad j = 1, 2, \ldots, n$$

i zadanie odwrotne

- RS - przy danych współczynnikach rzeczyw. a_j, $j = 0, 1, \ldots, n+1$, i b_j, $j = 1, 2, \ldots, n$, wyznaczamy wartości

$$y_k = \frac{a_0}{2} + \sum_{j=1}^{n} \left(a_j \cos \frac{\pi jk}{n+1} + b_j \sin \frac{\pi jk}{n+1} \right) + \frac{a_{n+1}}{2} (-1)^k$$

dla $k = 0, 1, \ldots, 2n+1$.

Algorytmy Goertzela i Reinscha I

Przy obliczaniu wartości wielomianu w postaci (1) w punkcie ξ należy wyznaczyć wartości wyrażeń:

$$C(\xi) = \sum_{j=0}^{n} d_j \cos j\xi \qquad S(\xi) = \sum_{j=1}^{n} d_j \sin j\xi$$

o rzeczyw. współ. d_j.
Można zauważyć, że $C(\xi)$ jest częścią rzeczywistą, a $S(\xi)$ częścią urojoną wartości wielomianu rzeczywistego $w(z) = \sum_{j=0}^{n} d_j z^j$ w punkcie zespolonym $\xi = \cos\xi + i \sin\xi$, tzn.

$$w(\xi) = C(\xi) + iS(\xi) \tag{5}$$

Do obliczenia $w(\xi)$ można zastsować alg. Hornera - wymaga w tym wypadku $4n - 2$ rzeczywistych mnożeń i $3n - 2$ rzeczywistych dodawań.
Istotnie mniejszy koszt ma *algorytm Goertzela*.Zauważmy, że wartość $w(\xi)$ jest równa reszcie z dzielenia w przez rzeczywisty trójmian kwadratowy
$(z - \xi)(z - \bar{\xi}) = z^2 - 2\cos\xi z + 1$ w punkcie $z = \xi$. Jeżeli bowiem

$$\sum_{j=0}^{n} d_j z^j = (z^2 - 2\cos\xi z + 1) \sum_{j=2}^{n} e_j z^{j-2} + e_1 z + e_0 \tag{6}$$

Algorytmy Goertzela i Reinscha II

to

$$w(\xi) = (e_1 \cos \xi + e_0) + i e_1 \sin \xi \tag{7}$$

Z równości (5) - (7) wynika algorytm Goertzela

Algorytm Goertzela

```
c := cos(ξ)
e_{n+1} := e_{n+2} := 0
for j:=n step -1 until 0 do
  e_j := d_j + 2c e_{j+1} − e_{j+2}
  C(ξ) := e_0 − c e_1
  S(ξ) := e_1 sin(ξ)
```

Algorytm może dawać niepoprawne wyniki dla $|\xi| \ll 1$. Wynika to stąd, że zadanie jest źle uwarunkowane dla pomocniczej zmiennej $c = cos(\xi)$, choć dla oryginalnej zmiennej ξ jest dobrze uwarunkowane.
Algorytm zmodyfikował Reinsch, i jest to algorytm numerycznie poprawny.

Algorytm Reinscha

$e_{n+2} := \delta e_{n+1} := 0$
if $cos(\xi) > 0$ then
  p := $-4sin^2(\xi/2)$
  for j:=n step -q until 0 do
    $e_{j+1} := \delta e_{j+1} + e_{j+2}$
    $\delta e_j := pe_{j+1} + \delta e_{j+1} + d_j$
else
  p := $4cos^2(\xi/2)$
  for j:=n step -1 until 0 do
    $e_{j+1} := \delta e_{j+1} - e_{j+2}$
    $\delta e_j := pe_{j+1} - \delta e_{j+1} + d_j$
end
$C(\xi) := \delta e_0 - pe_1/2$
$S(\xi) := e_1 sin(\xi)$

Można sprawdzić, że algorytm Reinscha wymaga $3n + 4$ rzeczywistych dodawań i tylko $n + 4$ rzeczywistych mnożeń (w alg. Hornera mnożeń było $4n - 2$).

Interpolacja funkcjami sklejanymi

Interpolacja funkcjami sklejanymi. Definicja zadania I

Niech (Δ) będzie układem punktów $x_0, x_1, \ldots, x_N$ dzielących przedział $[a, b]$ na N części:

$$(\Delta)\ a = x_0 < x_1 < \ldots x_N = b$$

W każdym z przedziałów $[x_i, x_{i+1}]$ przybliżamy f wielomanem ustalonego (zwykle niskiego) stopnia.
Istotne jest, aby przyjęte rozwiązanie w danym podprzedziale było ciągłe wraz z odpowiednimi pochodnymi na całym odcinku $[a, b]$. Takie własności mają *funkcje sklejane, ang. spline*.

Definicja funkcji sklejanej

Funkcją rzeczywistą S nazywamy *funkcją sklejaną* stopnia m z węzłami

$$(\Delta)\ a = x_0 < x_1 < \ldots x_N = b$$

jeżeli

- w każdym przedziale (x_{i-1}, x_i) dla $i = 0, 1, \ldots, N+1$, gdzie $x_{-1} := -\infty$, $x_{N+1} := +\infty$, S jest wielomianem stopnia nie wyższego niż m,
- S i jej pochodne rzędu $1, 2, \ldots, m-1$ są ciągłe na całym zbiorze rzeczywistym, $S \in C^{m-1}$.

Interpolacja funkcjami sklejanymi. Definicja zadania II

W naprostszym przypadku, gdy $m = 1$, funkcja sklejana jest łamaną.
Z definicji wynika, że wielomiany są szczególnym przypadkiem funkcji sklejanych.

Splajn pierwszego stopnia

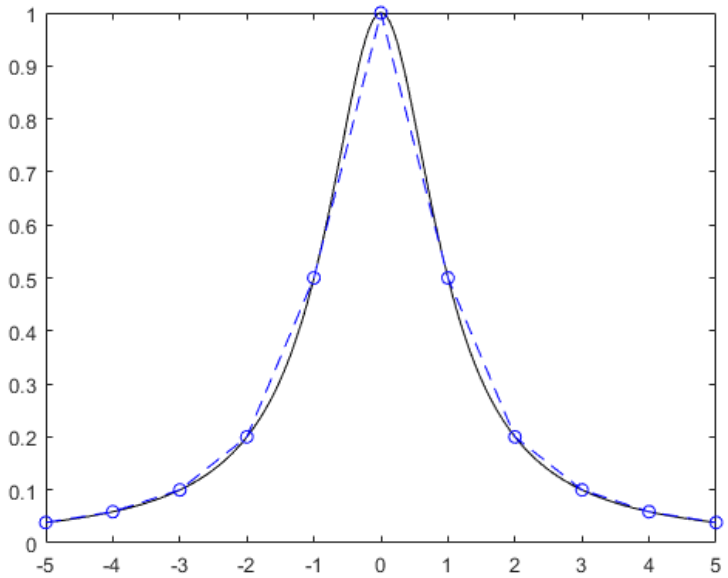
Przypadek najprostszy

- Funkcje sklejane pierwszego stopnia $\rightarrow$ interpolacja liniowa pomiędzy węzłami x_i i x_{i+1}. Wykorzystując postać Newtona otrzymujemy:

$$y(x) = S_i(x) = y_i + \frac{(y_{i+1} - y_i)}{(x_{i+1} - x_i)}(x - x_i) \qquad x_i \leqslant x \leqslant x_{i+1}$$

- Sformułowanie to spełnia warunek $S_i(x_{i+1}) = S_{i+1}(x_{i+1})$

Przykład: funkcja $y = 1/(1 + x^2)$ interpolowana splajnami pierwszego stopnia



Splajny trzeciego stopnia (kubiczne)

- Wielomian stopnia n jest definiowany jednoznacznie przez $n+1$ równań $\rightarrow$ kolejne równania można otrzymać z narzucenia warunku ciągłości pierwszej i kolejnych pochodnych
- Jest to sposób na wygładzenie krzywej interpolującej
- Splajn trzeciego stopnia w każdym z n przedziałów jest postaci: $S_i(x) = a_0^i + a_1^i x + a_2^i x^2 + a_3^i x^3$, $i = 0 \ldots n-1 \rightarrow$ potrzebnych jest $4n$ różnych warunków, żeby wyznaczyć współczynniki wszystkich splajnów
- Warunek interpolacji funkcjami sklejanymi z definicji daje dokładnie 2 równania dla każdego splajnu, czyli $2n$ równań postaci:

$$S_i(x_i) = y_i, \; S_i(x_{i+1}) = y_{i+1}, \quad i = 0 \ldots n-1$$

- Brakujące warunki otrzymujemy obliczając pierwsze i drugie pochodne i przyrównując je w połączeniach splajnów:

$$S_i^{'}(x) = \frac{dS_i(x)}{dx} = a_1^i + 2a_2^i x + 3a_3^i x^2 \qquad S_i^{'}(x_i) = S_{i-1}^{'}(x_i), \quad i = 1 \ldots n-1$$
$$S_i^{''}(x) = \frac{d^2 S_i(x)}{dx^2} = 2a_2^i + 6a_3^i x \qquad S_i^{''}(x_i) = S_{i-1}^{''}(x_i), \quad i = 1 \ldots n-1$$

  Pozostałe 2 warunki dla węzłów brzegowych można narzucić np. w postaci $S_0^{''}(x_0) = S_{n-1}^{''}(x_n) = 0$
- Otrzymany układ $4n$ równań tworzy macierz przekątniową pasmową. Zadanie można rozwiązać przy użyciu np. metod eliminacji.

Przykładowe zadania i obliczenia

Interpolacja Lagrange'a. Obliczanie wartości funkcji I

Oblicz wartości poniższych funkcji dla zadanych wartości wykorzystując interpolację Lagrange'a

- $f(x) = \sqrt{x}$ dla $x = 6$
- $f(x) = \log_{10} x$ dla $x = 40$
- $f(x) = \sqrt[3]{x}$ dla $x = 16$

Interpolacja Lagrange'a. Obliczanie wartości funkcji I

Rozważmy przykład $f(x) = \sqrt{x}$ dla $x = 6$

1. Wybieramy punkty, co najmniej 3, dla których wartość funkcji $f(x)$ jest znana i takie, żeby punkt $x = 6$ leżał w środku wybranego przedziału: $\{x, f(x)\}$: $\{1, 1\}$, $\{4, 2\}$, $\{9, 3\}$
2. Tworzymy z definicji wielomian interpolacyjny Lagrange'a

$$L_2(x) = f(x_0)\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} + f(x_1)\frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} + f(x_2)\frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}$$

3. Obliczamy wartość dla $x = 6$

$$L_2(6) = 1\frac{(6-4)(6-9)}{(1-4)(1-9)} + 2\frac{(6-1)(6-9)}{(4-1)(4-9)} + 3\frac{(6-1)(6-4)}{(9-1)(9-4)} =$$
$$= -\frac{1}{4} + 2 + \frac{3}{4} = 2\frac{1}{2}$$

Interpolacja Lagrange'a. Postać Newtona I

- Postać Newtona wymaga obliczenia ilorazów różnicowych, które są równe współczynnikom postaci Newtona $b_k = f[x_0, x_1, \ldots, x_k]$ oraz zachodzi:

$$f[x_0] = f(x_0)$$
$$f[x_i, x_{i+1} \ldots, x_{i+k}] = \frac{f[x_{i+1},\ldots,x_{i+k}] - f[x_i,\ldots,x_{i+k-1}]}{x_{i+k} - x_i}$$

- Oblicz ilorazy różnicowe dla funkcji, o której wiemy, że przyjmuje następujące wartości:
$(x, f(x))$: $(1,1)$, $(2,5)$, $(3,-6)$, $(5,2)$
Iloraz różnicowy I rzędu:

$$f[1,2] = \frac{f[2]-f[1]}{2-1} = \frac{5-1}{1} = 4$$
$$f[2,3] = \frac{f[3]-f[2]}{3-2} = \frac{-6-5}{1} = -11$$
$$f[3,5] = \frac{f[5]-f[3]}{5-3} = \frac{2+6}{2} = 4$$

Iloraz różnicowy II rzędu:

$$f[1,2,3] = \frac{f[2,3]-f[1,2]}{3-1} = \frac{-11-4}{2} = -7\frac{1}{2}$$
$$f[2,3,5] = \frac{f[3,5]-f[2,3]}{5-2} = \frac{4+11}{3} = 5$$

Iloraz różnicowy III rzędu:

$$f[1,2,3,5] = \frac{f[2,3,5] - f[1,2,3]}{5-1} = \frac{5 + 7\frac{1}{2}}{4} = 3\frac{1}{8}$$

Interpolacja Lagrange'a. Postać Newtona II

- Podaj postać Newtona $W(x) = \sum_{k=0}^{2} b_k p_k(x)$ dla wielomianu W w postaci naturalnej $W(x) = 2x^2 - 4x + 3$ oraz punktów $x_0 = 1$, $x_1 = 2$, $x_2 = 3$
  Konstruujemy wielomiany p_i:

$$\begin{aligned} p_0(x) &= 1 \\ p_1(x) &= (x - x_0) = (x - 1) \\ p_2(x) &= (x - x_0)(x - x_1) = (x - 1)(x - 2) \end{aligned}$$

  Obliczamy współczynniki b_i

$$\begin{aligned} b_0 &= f[x_0] = w(1) = 1 \\ b_1 &= f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0} = \frac{w(2) - w(1)}{2 - 1} = \frac{3 - 1}{1} = 2 \\ b_2 &= f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{\frac{f[x_2] - f[x_1]}{x_2 - x_1} - 2}{3 - 1} = \frac{\frac{w(3) - w(2)}{3 - 2} - 2}{2} = 2 \end{aligned}$$

  Szukana postać Newtona wielomianu W:

$$W(x) = 1 + 2(x - 1) + 2(x - 1)(x - 2)$$

Reszta interpolacyjna I

- Reszta interpolacyjna r to
$$r(x) = f(x) - L_n(x)$$
- dokładna wartość funkcji f w punkcie x nie jest znana, dlatego korzystamy z twierdzenia
Istnieje punkt $\xi = \xi(x) \in [a, b]$ taki, że
$$r(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} p_{n+1}(x)$$
przy czym zakładamy, że f ma ciągłą pochodną $(n+1)$. rzędu na przedziale $[a, b]$ zawierającym punkty x_0, x_1, ..., x_n oraz x
- Jaki jest błąd przy obliczaniu wartości funkcji cos dla $5\pi/8$, jeśli znane są wartości tej funkcji dla 0, $\pi/6$, $\pi/4$, $\pi/3$, $\pi/2$, $2\pi/3$, π
Szacujemy

$$r(\tfrac{5\pi}{8}) = \frac{\cos^{(7)}(\xi(x))}{7!} \left(\tfrac{5\pi}{8} - 0\right)\left(\tfrac{5\pi}{8} - \tfrac{\pi}{6}\right)\left(\tfrac{5\pi}{8} - \tfrac{\pi}{4}\right)\left(\tfrac{5\pi}{8} - \tfrac{\pi}{3}\right)\left(\tfrac{5\pi}{8} - \tfrac{\pi}{2}\right)\left(\tfrac{5\pi}{8} - \tfrac{2\pi}{3}\right)\left(\tfrac{5\pi}{8} - \pi\right) \leqslant \frac{\pi^7}{7!}$$

Postać Newtona I

- Wielomian Hermite'a w postaci Newtona

$$H_n(x) = \sum_{l=0}^{n} b_l p_l(x) = \sum_{i=0}^{k} \sum_{j=0}^{m_i-1} b_{s(i)+j} p_{s(i)+j}(x)$$

- Skonstruuj wielomian 4. stopnia W taki, że
  $W(0) = 1,\ W'(0) = 2,\ W(2) = 2,\ W'(2) = 1,\ W''(2) = 3$
  Obliczamy indeksy

$$\begin{aligned} l = 0 &= s(0) + 0 \\ l = 1 &= s(0) + 1 \\ l = 2 &= s(1) + 0 \\ l = 3 &= s(1) + 1 \\ l = 4 &= s(1) + 2 \end{aligned}$$

  Konstruujemy wielomiany p_i

$$\begin{aligned} p_0(x) &:= 1 \\ p_1(x) &= p_{s(0)+1}(x) = (x - x_0) = x \\ p_2(x) &= p_{s(1)+0}(x) = (x - x_0)^{m_0}(x - x_1)^0 = (x - 0)^2 = x^2 \\ p_3(x) &= p_{s(1)+1}(x) = (x - x_0)^{m_0}(x - x_1)^1 = (x - 0)^2(x - 2) = x^2(x - 2) \\ p_4(x) &= p_{s(1)+2}(x) = (x - x_0)^{m_0}(x - x_1)^2 = (x - 0)^2(x - 2)^2 = x^2(x - 2)^2 \end{aligned}$$

  Obliczamy współczynniki b_i (odpowiednie ilorazy różnicowe)

Postać Newtona II

$$b_0 = b_{s(0)+0} = f[x_0, 1] = f[x_0] = W(x_0) = 1$$
$$b_1 = b_{s(0)+1} = f[x_0, 2] = \frac{f'(x_0)}{1!} = W'(0) = 2$$
$$b_2 = b_{s(1)+0} = f[x_0, 2, x_1, 1] = \frac{f[x_0,1,x_1,1]-f[x_0,2]}{x_1-x_0} = \frac{f[x_1,1]-f[x_0,1]}{x_1-x_0}\frac{1}{2} - W'(0)\frac{1}{2} = \frac{1}{4} - 1 = -\frac{3}{4}$$
$$b_3 = b_{s(1)+1} = f[x_0, 2, x_1, 2] = \frac{f[x_0,1,x_1,2]-f[x_0,2,x_1,1]}{x_1-x_0} = \frac{f[x_1,2]-f[x_0,1,x_1,1]}{x_1-x_0}\frac{1}{2} + \frac{3}{8} = \frac{1-\frac{1}{2}}{2}\frac{1}{2} + \frac{3}{8} =$$
$$= \frac{1}{8} + \frac{3}{8} = \frac{1}{2}$$
$$b_4 = b_{s(1)+2} = f[x_0, 2, x_1, 3] = \frac{f[x_0,1,x_1,3]-f[x_0,2,x_1,2]}{x_1-x_0} = \frac{f[x_1,3]-f[x_0,1,x_1,2]}{x_1-x_0}\frac{1}{2} - \frac{1}{2}\frac{1}{2} =$$
$$= \left(\frac{3}{2} - \frac{f[x_1,2]-f[x_0,1,x_1,1]}{x_1-x_0}\right)\frac{1}{4} - \frac{1}{4} =$$
$$= \left[\frac{3}{2} - \frac{1}{2}\left(1 - \frac{1}{2}\right)\right]\frac{1}{4} - \frac{1}{4} = \frac{5}{16} - \frac{1}{4} = \frac{1}{16}$$

Szukany wielomian W jest postaci:

$$W(x) = 1 + 2x - \frac{3}{4}x^2 + \frac{1}{2}x^2(x-2) + \frac{1}{16}x^2(x-2)^2$$